Inference Acceleration as Closed-Loop Perturbation: Sensitivity-Guided Speedups for VLA Policies

Anonymous Authors

Abstract

Inference acceleration for vision-language-action (VLA) policies is often treated as a systems problem: quantize weights, compile graphs, fuse kernels, or replay static execution paths, then report drift, success, latency, and memory. This framing is incomplete for robot policies. A VLA model is a closed-loop controller, so an implementation perturbation changes not only the current action but also the future state distribution induced by interaction with the environment. We formulate post-training VLA acceleration as a closed-loop policy perturbation problem and derive three practical consequences: acceleration error is filtered by feedback dynamics and task margins; open-loop action drift is insufficient without matched closed-loop rollouts; and sensitivity is anisotropic across action channels, rollout phases, and model boundaries. On a GR00T N1.5 policy in LIBERO-10, selective W4A8 fake quantization remains in the FP16 behavioral range but redistributes which rollouts succeed. Speed-only action-head compilation gives a 2.23× median serving-latency speedup on a 33-case probe but reduces success from 19/33 to 16/33. Sensitivity-guided protection finds a strong local tactic: keeping only policy steps 0–120 in eager mode restores 19/33 success while preserving speed-only latency. Held-out validation changes the conclusion from selecting one fixed winner to selecting tactics by worst-fold success, paired regressions, and speed: across two 30-case folds, 0–120 and a combined early-block plus 0–120 tactic both reach 47/60 pooled success, while behavior-first and speed-constrained objectives choose different tactics. The result is not a claim of final packed-kernel deployment efficiency; it is a robotics evaluation and design guide for acceleration under closed-loop sensitivity.

1 Introduction

Large vision-language-action models increasingly serve as generalist robot policies. They map visual observations and language instructions into action chunks, often through diffusion-style denoising or transformer action heads [2–4, 7, 10]. As these models grow, deployment systems will quantize weights, compile graphs, fuse kernels, replay static CUDA graphs, cache repeated structure, or mix high- and low-precision execution. The usual question is whether the accelerated implementation is numerically close to the reference model and faster to serve.

For closed-loop robot policies, numerical closeness is not the whole problem. An action difference at time t changes the next observation; the next action is then produced on a state that may not have been visited by the original FP16 policy. In manipulation, this feedback can change contact timing, grasp stability, object pose, or whether the terminal state crosses a binary success predicate. A tiny implementation difference can be absorbed in one phase, amplified near a contact boundary, or even repair a failure by nudging the rollout into a better trajectory basin.

This paper studies the following question: how should VLA inference acceleration be designed and evaluated when the accelerated implementation is part of a closed-loop control system? Our answer is that post-training acceleration should be treated as a structured policy perturbation. Quantization, graph compilation, eager islands, and kernel replacement are implementation maps from a reference policy to a constrained, faster policy. The relevant object is not only the one-step action drift on fixed observations, but also how the implementation redistributes success and failure over matched closed-loop rollouts.

We make three contributions:

1. We formulate VLA inference acceleration as a closed-loop policy perturbation problem and derive local propagation, margin-crossing, and distribution-shift expressions that explain why open-loop drift is not sufficient.

2. We provide a GR00T/LIBERO behavior study showing that selective W4A8 fake quantization and action-head compilation both create paired repairs and regressions rather than monotonic improvements.

3. We demonstrate sensitivity-guided acceleration and multi-fold tactic selection: a narrow early eager window is a strong local proxy, but held-out folds require selecting from a speed–risk Pareto frontier rather than trusting one winner.

The contribution is not a single deployment backend. Instead, we use quantization and compilation probes to expose the geometry of the problem: which perturbations are absorbed, which are amplified, which rollouts are repaired or regressed, and which dimensions, phases, or modules should be protected. The resulting design

rule is simple: identify closed-loop-sensitive dimensions, durations, and modules, then choose tactics by held-out paired repair/regression and speed rather than by a single local probe.

2 Related Work

VLA policies. RT-1 and RT-2 showed that large transformer policies can scale robot manipulation and transfer visual-language knowledge into action [2, 3]. OpenVLA released an open-source VLA trained on large robot mixtures [7]. GR00T N1 uses a vision-language module to condition a diffusion transformer action module [10]. Diffusion Policy and related action-diffusion methods model robot actions as conditional denoising under receding-horizon control [4]. Our work focuses on what happens when these closed-loop policies are accelerated after training.

Post-training quantization and acceleration. LLM.int8(), GPTQ, SmoothQuant, and AWQ show that transformer quantization can preserve quality when calibration accounts for activation structure [5, 6, 8, 12]. QuantVLA targets PTQ for VLA models with selective quantization, attention temperature matching, and output head balancing [13]. These methods motivate the systems side of the problem. We study the complementary robotics question: how small acceleration-induced action differences change closed-loop rollout behavior.

Sequential prediction and feedback. The gap between offline error and closed-loop behavior is classic in imitation learning and sequential prediction. DAgger formalizes how policies trained or evaluated only under a fixed data distribution can suffer from compounding errors [11]. Feedback control similarly emphasizes that closed-loop behavior depends on controller perturbations, plant dynamics, and feedback gain [1]. We apply this lens to inference acceleration.

3 Closed-Loop Perturbation View

Let $\pi_\theta(a \mid s, l)$ denote a reference FP16 VLA policy conditioned on observation s and language l. A post-training acceleration transform constructs an implemented policy

$$\varphi = \mathcal{A}(\theta; C, S), \qquad \pi_q = \pi_\varphi, \tag{1}$$

where $\mathcal{A}$ may represent quantization, compilation, mixed precision, kernel replacement, or graph replay; C denotes calibration data; and S denotes systems choices such as graph boundaries or shape buckets. Even without gradient updates, this transform changes the implemented policy function.

3.1 Sensitivity Propagation

For local dynamics

$$s_{t+1} = f(s_t, a_t), \qquad a_t = \pi(s_t, l), \tag{2}$$

let $\eta_t = \pi_q(s_t, l) - \pi_\theta(s_t, l)$ be the local acceleration-induced action perturbation on the nominal FP16 trajectory. Let $A_t = \partial f/\partial s$, $B_t = \partial f/\partial a$, $K_t = \partial \pi_\theta/\partial s$, and $F_t = A_t + B_t K_t$. Under a local first-order approximation,

$$\delta s_T \approx \sum_{t=0}^{T-1} \left(F_{T-1} F_{T-2} \cdots F_{t+1}\right) B_t \eta_t. \tag{3}$$

Thus acceleration error is not determined by $\|\eta_t\|$ alone. Its effect depends on direction, dynamics, policy feedback, and future closed-loop gain.

3.2 Margin Crossing

Let $h(s_T)$ be an implicit terminal success margin, with success when $h(s_T) > 0$. For a small terminal perturbation,

$$h(s_T^q) - h(s_T) \approx \nabla h(s_T)^\top \delta s_T. \tag{4}$$

A rollout can flip when

$$\left|\nabla h(s_T)^\top \delta s_T\right| \gtrsim |h(s_T)|. \tag{5}$$

The sign determines whether the perturbation repairs a failure or regresses a success. This explains why two rollouts with similar one-step action drift can have different outcomes: high-margin trajectories absorb perturbations, while low-margin trajectories near contact or placement boundaries can cross the success threshold.

3.3 Open-Loop Drift Is Insufficient

Offline validation estimates a loss $\ell(s)$ under a fixed distribution, typically d_{π_θ} or a dataset distribution. Closed-loop accelerated execution samples from d_{π_q}:

$$\mathbb{E}_{d_{\pi_q}}[\ell] = \mathbb{E}_{d_{\pi_\theta}}[\ell] + \left(\mathbb{E}_{d_{\pi_q}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell]\right). \tag{6}$$

Open-loop action drift controls only the fixed-distribution term. It does not control how far the accelerated policy moves the visited state distribution. This motivates matched closed-loop rollouts and paired repair/regression accounting.

3.4 Anisotropic Sensitivity

Equations (3) and (5) induce a sensitivity-weighted local objective

$$\min_Q \ \mathbb{E} \sum_t \eta_t(Q)^\top G_t \eta_t(Q), \tag{7}$$

where G_t summarizes downstream dynamics, feedback, and terminal-margin sensitivity. The exact G_t is difficult to estimate, but the expression identifies the structure: action channels, rollout phases, and model boundaries need not be equally sensitive. A useful acceleration candidate should have high speed benefit and low sensitivity-weighted risk.

4 Experimental Setup

Model and benchmark. We use the GR00T N1.5 LIBERO long-horizon checkpoint and evaluate on LIBERO-10 [9, 10]. The accepted FP16 baseline matches the official reference result of 38/50 on the standard 5-trial LIBERO-10 run. The policy predicts action chunks with seven action components: x, y, z, roll, pitch, yaw, and gripper. Unless noted otherwise, experiments use 8 denoising steps.

Evidence tracks. Track A applies selective W4A8 fake quantization and optional attention/output compensation to the same FP16 policy, then evaluates open-loop drift and closed-loop rollout redistribution. Track B uses controlled action perturbations, first-divergence traces, and `torch.compile` action-head boundaries to identify sensitive action channels, rollout phases, and model boundaries. Track C keeps weights in FP16, compiles the action head for speed, and inserts small eager islands or duration windows as protected regions.

Quantization and compensation. We study the selective `llm_dit_mlp` W4A8 fake-quantization scope: 84 LLM linear layers and 32 DiT MLP linear layers, for 116 quantized modules. DiT attention projections remain in floating point. Following QuantVLA terminology, we evaluate attention temperature matching (ATM) and output head balancing (OHB) [13]. Our implementation should be read as a reproduction-oriented approximation, not a claim of exact code-level equivalence.

Closed-loop protocol. The quantization analysis evaluates 150 episodes: 10 LIBERO-10 tasks and 15 initial states per task. The sensitivity-guided acceleration search starts with a 33-case matched probe spanning tasks 4, 6, and 8, including easy successes, speed-only regressions, and beneficial speed-only branches. We then validate tactic selection on two additional 30-case folds over all LIBERO-10 tasks, using initial states 15–17 and 18–20. Every mode is evaluated on the same task-initialization pairs with deterministic policy seeds. We report success, paired repair/regression, server p50 latency, eager-request fraction, first-divergence diagnostics, worst-fold success, and worst-fold speedup.

5 Results

5.1 W4A8 Changes Behavior Without Leaving the FP16 Range

Table 1 summarizes the 150-episode quantization track. Selective W4A8 remains in the FP16 behavioral range, and some quantized variants have higher point estimates. These gaps should not be interpreted as evidence that quantization is inherently better than FP16. The paired outcomes show substantial trajectory redistribution.

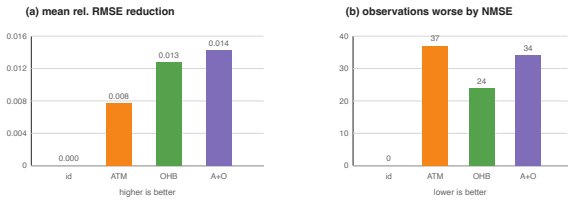

Figure 1: Offline mean drift versus observation-level regressions. Mean drift can improve while individual held-out observations regress.

Table 1: Closed-loop LIBERO-10 success over 150 matched task-initialization pairs.

Policy	Successes
FP16	108/150
W4A8 `llm_dit_mlp` + none	113/150
W4A8 `llm_dit_mlp` + ATM	114/150
W4A8 `llm_dit_mlp` + OHB	116/150
W4A8 `llm_dit_mlp` + ATM+OHB	114/150

Offline drift is also incomplete. ATM+OHB has the best mean held-out action drift, but it still worsens 34 of 128 held-out observations by NMSE relative to uncompensated W4A8. In closed loop, OHB has the highest aggregate point estimate, while ATM+OHB does not. This is consistent with Equation (6): fixed-observation drift does not measure the accelerated policy's induced state distribution.

5.2 Paired Rollouts Expose Repairs and Regressions

Aggregate success hides behavioral churn. OHB is not simply "3 episodes better" than uncompensated W4A8: it repairs 16 failures and introduces 13 new failures. The net gain is small, but the underlying trajectory redistribution is substantial.

Table 2: Paired outcomes over the same 150 task-initialization pairs.

Comparison	Repair	Regress	SS	FF	Net
ATM vs none	14	13	100	23	+1
OHB vs none	16	13	100	21	+3
OHB vs ATM	15	13	101	21	+2

Keyframe inspection confirms that these flips are visible trajectory-basin changes, not only final-frame threshold noise. Repaired cases often branch early and terminate much sooner than the corresponding failures. Regressed cases show the complementary pattern: a small perturbation disrupts an early object relation or produces a stable but insufficiently progressive path.

Figure 2: Paired closed-loop rollout flips. Net success-rate changes decompose into repaired failures, new regressions, unchanged successes, and unchanged failures.

Table 3: Controlled action perturbations on two FP16-success cases. S/F denotes success/failure and policy requests.

Probe	Window	Outcomes
No perturbation	Full	S224; S649
6D continuous	Full	F991; F991
x only	Full	S349; F991
y only	Full	F991; F991
z only	Full	S239; S666
Roll only	Full	F991; F991
Pitch only	Full	F991; F991
Yaw only	Full	F991; F991
y task 4	0:75 / 75:150 / 150:225	F991; F991; S219
y task 6	0:200 / 200:450 / 450:700	F991; S697; S752

5.3 Not All Dimensions, Durations, or Layers Are Equal

To isolate sensitivity from a specific quantizer, we ran controlled probes on two FP16-success cases. All full-horizon single-channel probes use the same continuous-action perturbation norm, $\|\delta a\|_2 = 0.03$. Table 3 shows that equal-norm perturbations have very different consequences. A z perturbation is absorbed in both cases, while y and rotation perturbations are catastrophic. Time also matters: the same y perturbation is damaging early but can be absorbed later.

Layer and graph boundaries are also sensitive. Compiling the action-head model gives a 2.21× speedup on two probe cases and succeeds on both, but leaving block 0 or block 1 as an eager island changes closed-loop outcomes despite nearly identical p50 latency. The result cannot be explained by speed alone; it is a layer/boundary sensitivity effect.

First-divergence traces show temporal separation: action differences appear first, millimeter-scale EEF differences appear later, and outcome flips occur much later through accumulated contact and progress changes. This supports the diagnostic principle: map sensitivity across action channels, rollout phases, and module boundaries before choosing what to quantize, compile, or protect.

5.4 Sensitivity-Guided Acceleration Search

We next test whether the sensitivity map can guide an engineering choice. The speed-oriented candidate compiles the action head. Protection candidates either keep selected layer blocks in eager mode or use a duration window in which the policy falls back to eager execution. Table 5 summarizes the 33-case discovery probe.

Table 4: Layer/boundary sensitivity under action-head compilation.

Execution mode	Success	T4:i9	T6:i8	p50
FP16 baseline	2/2	S224	S649	160.53 ms
Compile action head	2/2	S222	S916	72.80 ms
Compile + block 0 eager	1/2	F991	S404	72.62 ms
Compile + block 1 eager	0/2	F991	F991	73.10 ms

Table 5: Layer and duration proxy search on the 33-case matched set. Server p50 is warm serving latency.

Mode	Success	p50	Speedup	Eager frac.
FP16 baseline	19/33	156.22 ms	1.00×	–
Speed-only compile	16/33	70.20 ms	2.23×	–
Blocks 0–3 eager	18/33	67.58 ms	2.31×	–
Duration 0–250	18/33	88.75 ms	1.76×	0.376
Duration 0–120	19/33	69.66 ms	2.24×	0.190
Duration 0–180	18/33	77.55 ms	2.01×	0.281
Duration 0–220	16/33	90.49 ms	1.73×	0.328
Duration 80–240	14/33	75.98 ms	2.06×	0.214
Duration 120–280	16/33	73.48 ms	2.13×	0.216
Duration 160–240	15/33	69.29 ms	2.25×	0.111
Duration 240–320	17/33	66.71 ms	2.34×	0.084

The key probe result is not that more eager execution is safer. Speed-only compilation is fast but loses three successes relative to FP16. A broad early duration fallback, steps 0–250, recovers two successes but costs latency. The finer sweep reveals a better local point: protecting only steps 0–120 restores FP16-level success, 19/33, while preserving speed-only latency. Longer windows are worse, so the sensitive region is narrow and early.

Paired repair/regression explains the aggregate. Relative to speed-only, the 0–120 window repairs seven cases and regresses four, for a net +3. Relative to FP16, it repairs two baseline failures and regresses two baseline successes. Thus the accelerated implementation is not simply restored to the FP16 point; it explores a nearby closed-loop solution space.

Table 6: Paired decomposition versus speed-only compilation.

Mode	Repair	Regress	Net
Duration 0–250	4	2	+2
Duration 0–120	7	4	+3
Duration 0–180	5	3	+2
Duration 0–220	4	4	0
Duration 80–240	3	5	-2
Duration 120–280	2	2	0
Duration 160–240	2	3	-1
Duration 240–320	2	1	+1

Table 7: Multi-fold tactic selection. Fold A uses init 15–17; Fold B uses init 18–20. Regressions are against FP16.

Tactic	A	B	Pool	WSucc	MSpd	WSpd	Reg
Speed	25/30	20/30	45/60	0.667	2.01×	1.75×	7
0–120	22/30	25/30	47/60	0.733	1.84×	1.71×	3
Combo	22/30	25/30	47/60	0.733	1.41×	1.07×	1

Held-out folds change the engineering conclusion. Speed-only is best on Fold A but unsafe on Fold B. The combined tactic has the fewest regressions, but its worst-fold speedup is only 1.07×. The 0–120 tactic is therefore preferable under a speed-constrained objective, while the combined tactic is preferable under a behavior-first objective. Sensitivity-guided acceleration should thus output a Pareto choice under a validation protocol, not a claim that one protected region is universally optimal.

6 Discussion

Why can acceleration improve a rollout? FP16 is not an oracle; it is a learned policy with fragile regions. A small implementation perturbation can move a trajectory away from a failure mode. This is especially plausible on weak task slices. Therefore, an aggregate improvement should not be read as "low precision is better" or "compilation is better." It means the perturbed implementation enters different trajectory basins, some of which are more favorable under the benchmark's finite initial states and success predicates.

Practical guide. The experiments suggest a workflow for VLA acceleration studies. First, build a speed-oriented candidate and measure both latency and matched closed-loop success. Second, decompose outcomes into paired repairs and regressions. Third, for fragile cases, run first-divergence traces and controlled probes to identify sensitive action channels, rollout windows, and graph boundaries. Fourth, protect the smallest candidate region that improves the repair/regression balance on the probe set. Finally, re-evaluate candidate tactics on multiple held-out matched folds and select from the Pareto frontier using explicit constraints such as worst-fold success, maximum regressions, and minimum speedup.

Algorithmic form. The resulting heuristic tactic search takes a candidate set of quantization scopes, compile boundaries, eager islands, fallback windows, or kernels. It first prunes by cheap systems and open-loop proxies, then evaluates a small frontier on fragile matched rollouts. The final selector is not a single probe winner: a behavior-first objective maximizes worst-fold success and minimizes paired regressions, while a speed-constrained objective maximizes speedup subject to explicit closed-loop risk constraints.

Sensitivity-guided implementation search. This is analogous to compiler tactic search, but with a more expensive validation target. A compiler can enumerate many operator implementations for a fixed shape, benchmark latency, check local numerical agreement, and choose the fastest valid tactic. VLA acceleration has a similar candidate space—quantization scopes, graph boundaries, eager islands, fallback windows, kernel replacements, and calibration choices—but each candidate must be judged by closed-loop behavioral risk. Exhaustive rollout evaluation is infeasible. The useful strategy is therefore to prune candidates with cheap static and open-loop proxies, run small matched proxy rollouts on fragile cases, and reserve larger held-out validation for the few candidates with the best speed/risk trade-off. The 0–120 duration window should be read in this way: not as a universal tactic, but as a sensitivity-pruned candidate that must still be scored across held-out folds.

Limitations. The quantization track uses fake quantization, not final packed int4/int8 kernels. The mixed-execution acceleration probes use prototype action-head compilation and eager fallbacks; they are diagnostic evidence, not a production deployment benchmark. Results are from one GR00T N1.5 checkpoint and LIBERO-10. The acceleration track uses a 33-case discovery probe and two 30-case held-out folds, which expose tactic instability but are not a statistically exhaustive benchmark. We do not run real-robot experiments.

7 Conclusion

Post-training inference acceleration of VLA policies should be viewed as closed-loop policy perturbation. Small action errors matter when they interact with feedback dynamics, task margins, and trajectory basins. In our GR00T/LIBERO study, selective W4A8 fake quantization remains behaviorally robust but redistributes rollout outcomes. Speed-only action-head compilation gives substantial latency gains but introduces regressions. Sensitivity-guided search finds useful protected regions, such as the early 0–120 policy-step window, but held-out validation shows that the final choice is a constrained tactic-selection problem. The main lesson is practical: VLA acceleration papers should report paired closed-loop rollouts, repair/regression decompositions, sensitivity probes, held-out tactic validation, and latency under the same protocol, not only offline drift, aggregate success, or raw serving speed.

References

[1] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL `https://fbsbook.org`.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale.

arXiv preprint arXiv:2212.06817, 2022. URL https://arxiv.org/abs/2212.06817.

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL https://arxiv.org/abs/2307.15818.

[4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL https://arxiv.org/abs/2303.04137.

[5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022. URL https://arxiv.org/abs/2208.07339.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL https://arxiv.org/abs/2210.17323.

[7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. URL https://arxiv.org/abs/2406.09246.

[8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL https://arxiv.org/abs/2306.00978.

[9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL https://arxiv.org/abs/2306.03310.

[10] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL https://arxiv.org/abs/2503.14734.

[11] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *arXiv preprint arXiv:1011.0686*, 2010. URL https://arxiv.org/abs/1011.0686.

[12] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. URL https://arxiv.org/abs/2211.10438.

[13] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026. doi: 10.48550/arXiv.2602.20309. URL https://arxiv.org/abs/2602.20309.